takes an unbounded number of parameters, but if you want to write such a template, or if you're just interested in how it all works, you need to know the details.

Just as variadic functions are declared with an ellipsis (`...`) in the function parameter list, variadic templates are declared with an ellipsis in the template parameter list:

```
template<typename ... ParameterPack>
class my_template
{};
```

You can use variadic templates for a partial specialization of a template too, even if the primary template isn't variadic. For example, the primary template for `std::packaged_task<>` (section 4.2.1) is just a simple template with a single template parameter:

```
template<typename FunctionType>
class packaged_task;
```

However, this primary template is never defined anywhere; it's just a placeholder for the partial specialization:

```
template<typename ReturnType,typename ... Args>
class packaged_task<ReturnType(Args...)>;
```

It's this partial specialization that contains the real definition of the class; you saw in chapter 4 that you can write `std::packaged_task<int(std::string,double)>` to declare a task that takes a `std::string` and a `double` as parameters when you call it and that provides the result through a `std::future<int>`.

This declaration shows two additional features of variadic templates. The first feature is relatively simple: you can have normal template parameters (such as `Return-Type`) as well as variadic ones (`Args`) in the same declaration. The second feature demonstrated is the use of `Args...` in the template argument list of the specialization to show that the types that make up `Args` when the template is instantiated are to be listed here. Actually, because this is a partial specialization, it works as a pattern match; the types that occur in